


# 《 Introduction to Embedded System 》

# Lab-6

**Major:** Software Engineering

**Name:** Md Tasnimul Hasan

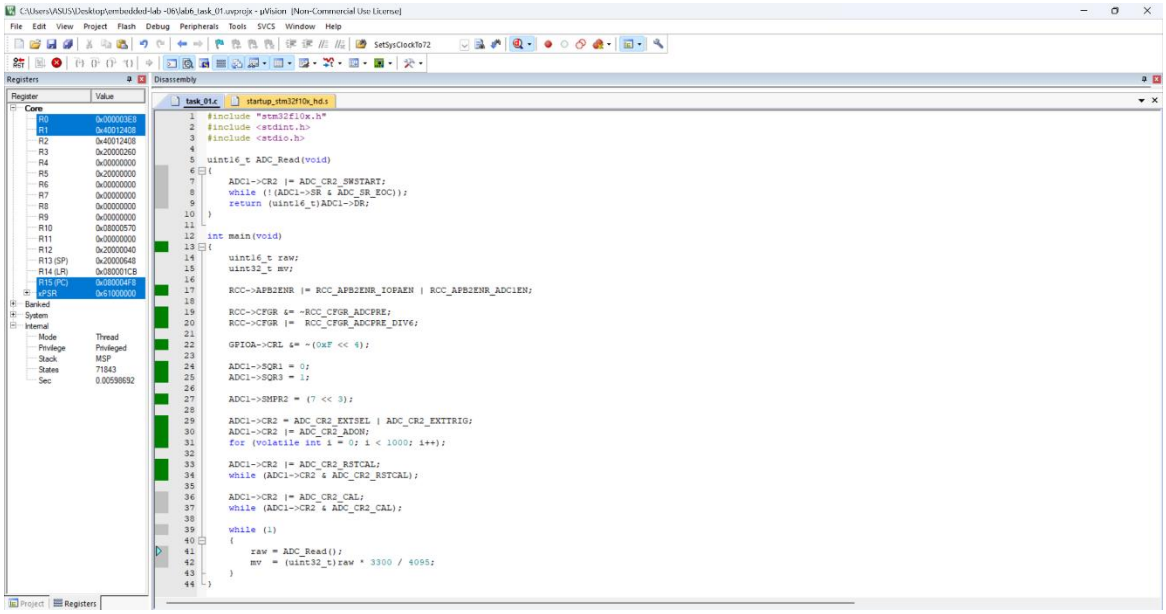
**Student ID:** 202480090104

## Task 1 – Single-channel ADC (PA1, channel 1)

**Objective:** To configure ADC1 of the STM32F103RC for single-conversion, software-triggered sampling on pin PA1 (channel 1), and to verify the 12-bit code-to-millivolt conversion formula.

**Description:** The GPIOA and ADC1 clocks are enabled and the ADC prescaler is set to divide-by-6 (12 MHz). PA1 is configured as a pure analog input with its 4-bit CRL field cleared and no pull resistor, channel 1 is selected as the only conversion (SQR1 = 0, SQR3 = 1) with a 239.5-cycle sampling time (SMPR2 = 0x38), and the ADC is powered on, calibrated (RSTCAL then CAL) and triggered by software. A blocking ADC_Read() function sets SWSTART, waits for EOC in SR, and returns DR. The raw code is converted using mV = code × 3300 / 4095 and tested at 0, 1024, 2048, 3072 and 4095, giving the expected 0, 825, 1650, 2475 and 3300 mV.

**Code and Output:**



```
#include "stm32f10x.h"
#include <stdint.h>
#include <stdio.h>

uint16_t ADC_Read(void)
{
    ADC1->CR2 |= ADC_CR2_SWSTART;
    while (!(ADC1->SR & ADC_SR_EOC));
    return (uint16_t)ADC1->DR;
}

int main(void)
{
    uint16_t raw;
    uint32_t mv;

    RCC->APB2ENR |= RCC_APB2ENR_IOPAEN | RCC_APB2ENR_ADC1EN;

    RCC->CFGR &= ~RCC_CFGR_ADCPRE;
    RCC->CFGR |=  RCC_CFGR_ADCPRE_DIV6;

    GPIOA->CRL &= ~(0xF << 4);

    ADC1->SQR1 = 0;
    ADC1->SQR3 = 1;

    ADC1->SMPR2 = (7 << 3);

    ADC1->CR2 = ADC_CR2_EXTSEL | ADC_CR2_EXTTRIG;
    ADC1->CR2 |= ADC_CR2_ADON;
    for (volatile int i = 0; i < 1000; i++);

    ADC1->CR2 |= ADC_CR2_RSTCAL;
    while (ADC1->CR2 & ADC_CR2_RSTCAL);

    ADC1->CR2 |= ADC_CR2_CAL;
    while (ADC1->CR2 & ADC_CR2_CAL);

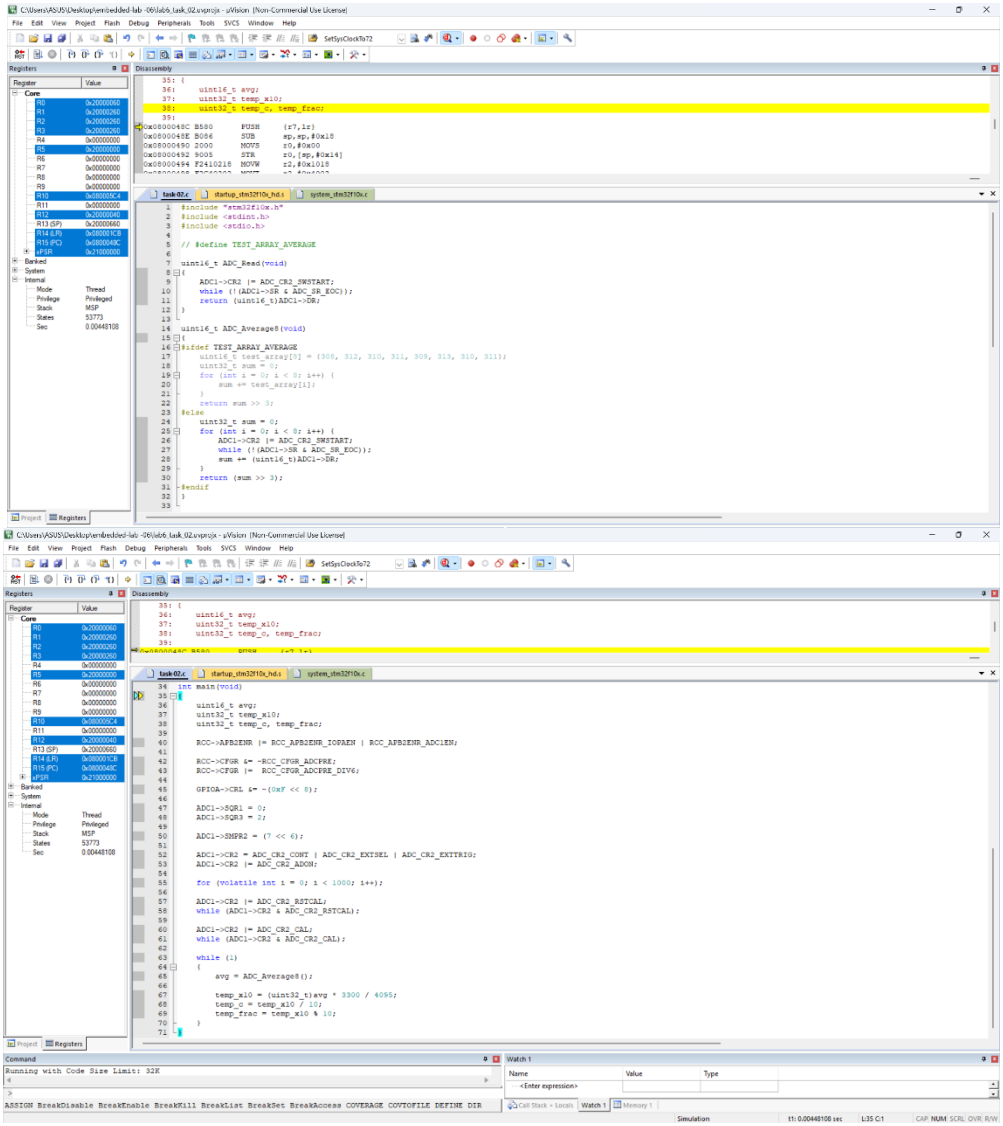
    while (1)
    {
        raw = ADC_Read();
        mv  = (uint32_t)raw * 3300 / 4095;
    }
}
```

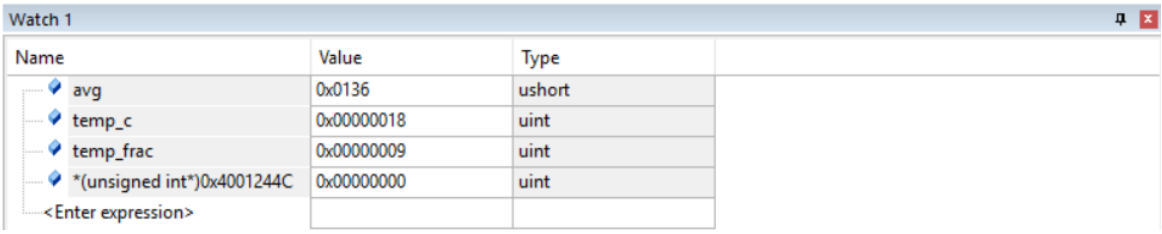
Watch 1

| Name | Value | Type |
|---|---|---|
| raw | 0x0400 | ushort |
| mv | 0x00000339 | uint |
| *(unsigned int*)0x40012408 | 0x00120005 | uint |
| *(unsigned int*)0x4001240C | 0x00000000 | uint |
| *(unsigned int*)0x4001244C | 0x00000400 | uint |
| *(unsigned int*)0x40012400 | 0x00000002 | uint |
| <Enter expression> | | |

**Task 2 – Continuous ADC with averaging (PA2, channel 2, LM35)**

**Objective:** To use ADC continuous-conversion mode together with an 8-sample averaging function to compute temperature from an LM35 sensor (10 mV/°C).
**Description:** The Task 1 setup is reused on PA2 (channel 2), with CONT added to CR2, re-calibration, and a single SWSTART to start free-running conversions (reading DR clears EOC). An ADC_Average8() function waits for EOC eight times, sums the readings and returns sum >> 3, which is then converted to tenths of °C via temp_x10 = avg × 3300 / 4095 and split into whole degrees and fraction. Averaging is validated with the array {308,312,310,311,309,313,310,311} (avg = 310 → 24.9 °C) and with forced DR values 311 → 25.0 °C and 1241 → 100.0 °C. Finally, three sampling times for channel 2 (1.5, 55.5 and 239.5 cycles) are checked in SMPR2 and their conversion times and maximum sample rates (≈1.17 µs/857 kSPS, 5.67 µs/176 kSPS, 21 µs/47.6 kSPS) are calculated.
**Code and Output:**



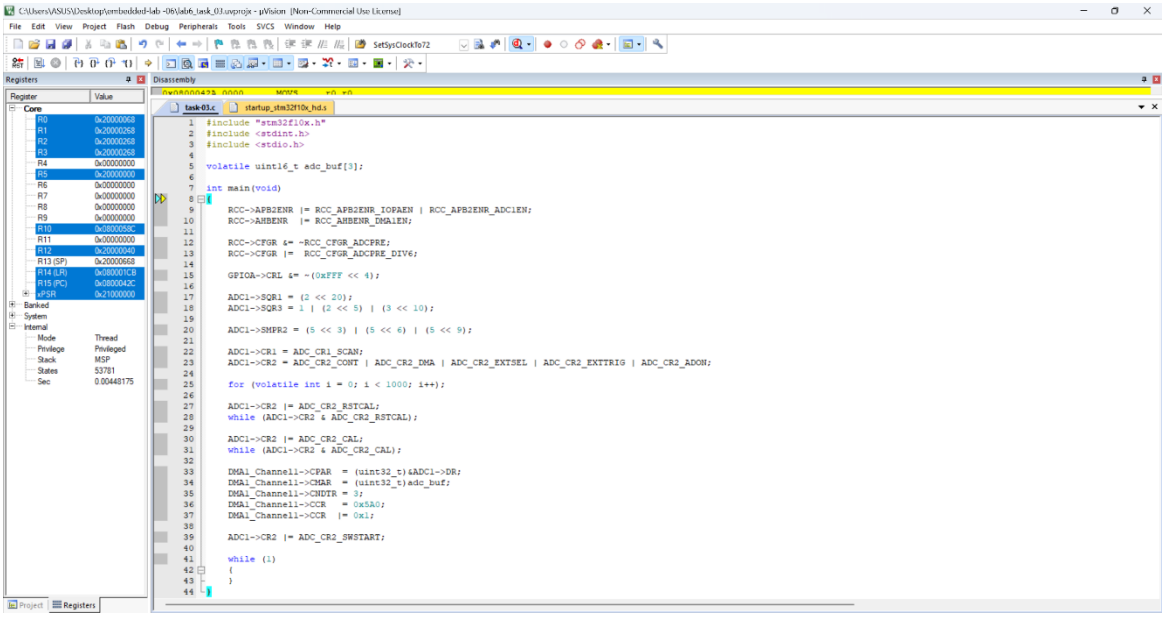


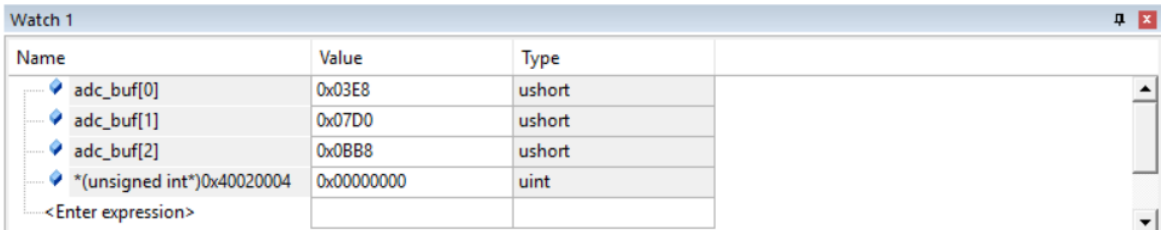

**Task 3 – Multi-channel scan with DMA (PA1/PA2/PA3)**

**Objective:** To configure a three-channel ADC scan sequence and transfer the results automatically into memory using DMA1 Channel 1 in circular mode.

**Description:** The GPIOA, ADC1 and DMA1 clocks are enabled, PA1–PA3 are set to analog by clearing CRL bits 15:4, and a 3-conversion sequence is built with SQR1 = (2 << 20) and SQR3 = 0x0C41 (channels 1, 2, 3), using 55.5-cycle sampling on all three channels (SMPR2 = 0x0B68). Scan mode is enabled in CR1, and CR2 is set to continuous + DMA + software trigger + ADON (0x001E0103) and calibrated. DMA1 Channel 1 is programmed with CPAR = &ADC1->DR, CMAR = adc_buf, CNDTR = 3 and CCR = 0x5A0 (memory increment, circular, 16-bit, peripheral-to-memory), and after SWSTART the buffer is checked to hold channels 1, 2 and 3 while CNDTR counts 3 → 0 → 3.

**Code and Output:**



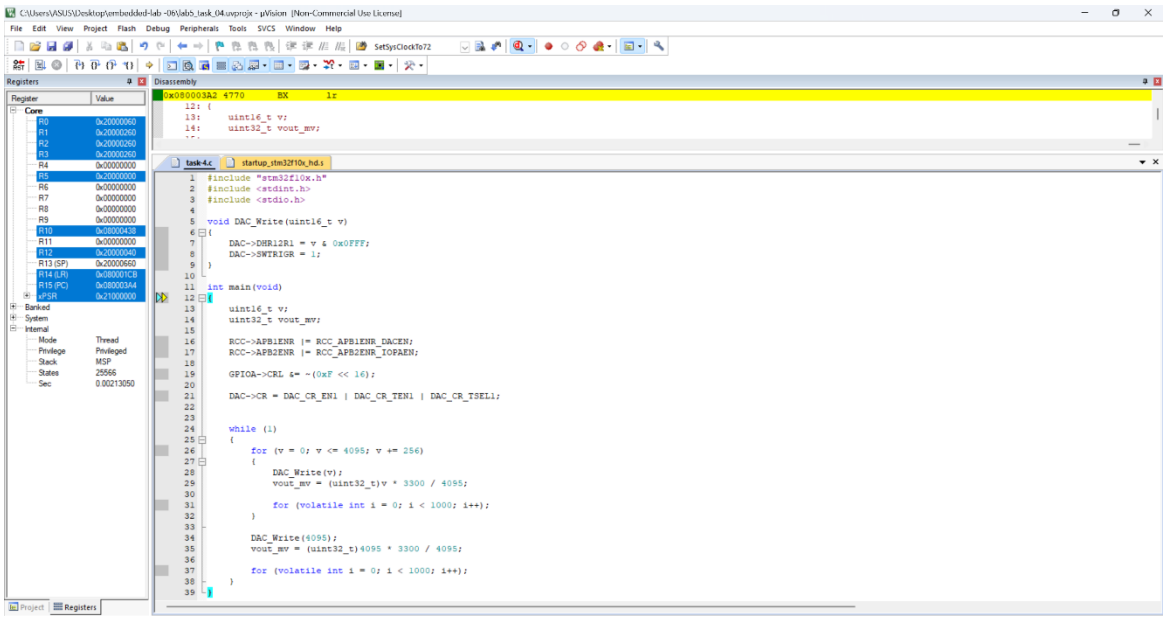



**Task 4 – DAC channel 1 output (PA4)**

**Objective:** To generate an analog output voltage on PA4 using DAC channel 1 with a software trigger, and to verify the DHR12R1 → DOR1 transfer and the effect of the output buffer.

**Description:** The DAC and GPIOA clocks are enabled, PA4 is configured as analog, and channel 1 is enabled with a software trigger (CR = 0x3D, TSEL = 111). A DAC_Write() function loads DHR12R1 with the 12-bit value and sets SWTRIGR, and in main() the code steps from 0 to 4095 in steps of 256 (finishing on 4095), storing vout_mv = v × 3300 / 4095 and confirming that DOR1 follows DHR12R1 after each trigger. The output points 0 → 0 mV, 1024 → 825 mV, 2048 → 1650 mV, 3072 → 2475 mV and 4095 → 3300 mV are checked against the chapter's "about 1.65 V" at mid-scale. Setting BOFF1 (bit 1) disables the output buffer (CR = 0x3F), and a table of 8-, 10- and 12-bit resolution (256, 1024, 4096 levels and their LSB sizes for a 3.3 V reference) is produced.

**Code and Output:**



```
#include "stm32f10x.h"
#include <stdint.h>
#include <stdio.h>

void DAC_Write(uint16_t v)
{
    DAC->DHR12R1 = v & 0x0FFF;
    DAC->SWTRIGR = 1;
}

int main(void)
{
    uint16_t v;
    uint32_t vout_mv;

    RCC->APB1ENR |= RCC_APB1ENR_DACEN;
    RCC->APB2ENR |= RCC_APB2ENR_IOPAEN;

    GPIOA->CRL &= ~(0xF << 16);

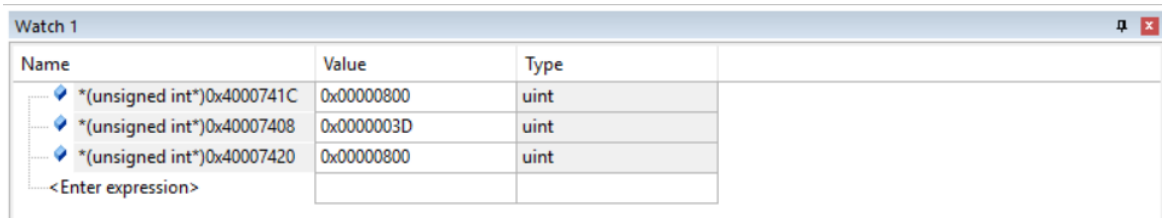
    DAC->CR = DAC_CR_EN1 | DAC_CR_TEN1 | DAC_CR_TSEL1;


    while (1)
    {
        for (v = 0; v <= 4095; v += 256)
        {
            DAC_Write(v);
            vout_mv = (uint32_t)v * 3300 / 4095;

            for (volatile int i = 0; i < 1000; i++);
        }

        DAC_Write(4095);
        vout_mv = (uint32_t)4095 * 3300 / 4095;

        for (volatile int i = 0; i < 1000; i++);
    }
}
```

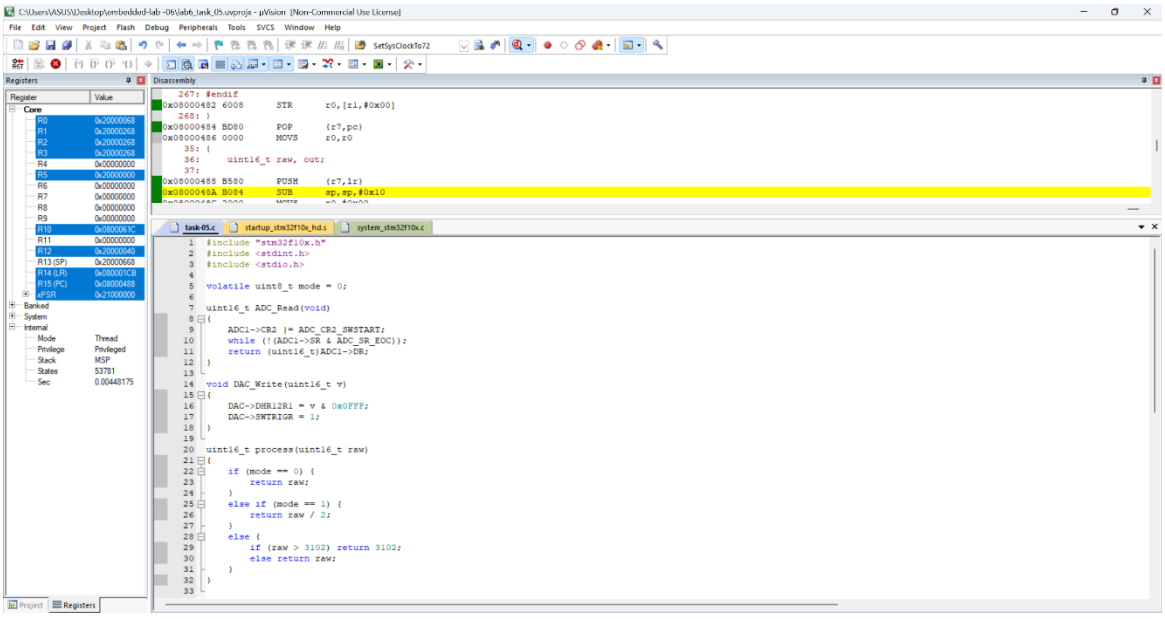


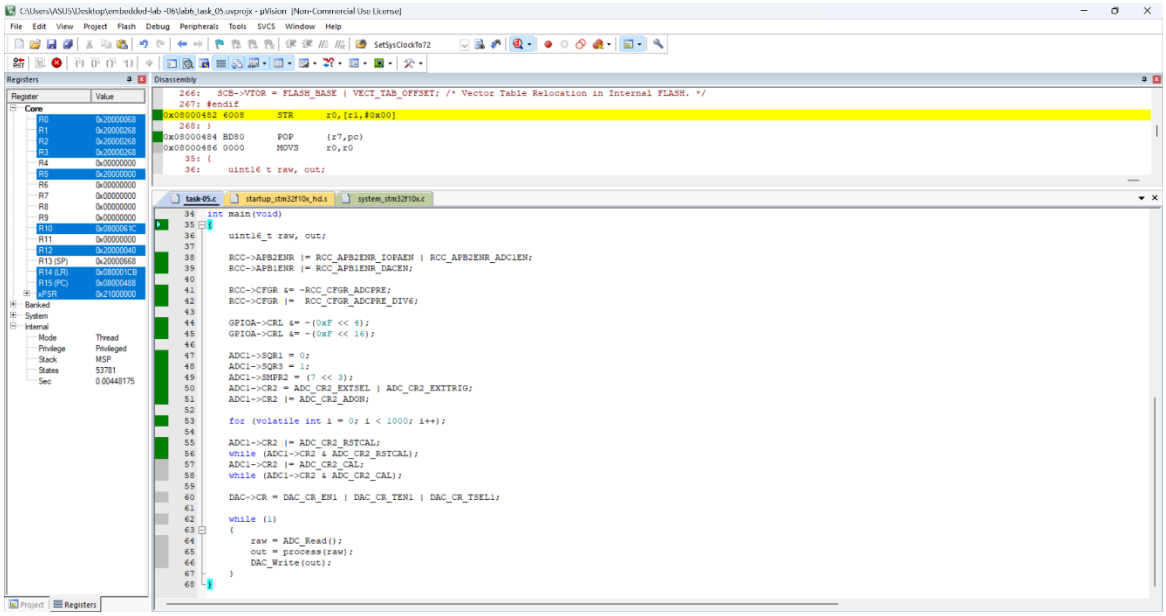

**Task 5 – ADC → processor → DAC loop**

**Objective:** To combine the ADC input (PA1) and DAC output (PA4) in a single project and implement a selectable processing mode (pass-through, half voltage, clipped) between them.

**Description:** The Task 1 ADC code and Task 4 DAC code are merged into one project and both peripherals are initialised, with a volatile mode variable and a process() function that returns raw (mode 0), raw / 2 (mode 1), or raw clipped to 3102 (mode 2, about 2.5 V since 2500 × 4095 / 3300 ≈ 3102). The main loop performs raw = ADC_Read(), out = process(raw) and DAC_Write(out), with mode, raw, out, ADC1->DR, DAC->DHR12R1 and DAC->DOR1 observed in the Watch window while mode is changed at run time. For each mode, ADC1->DR is forced to 1000, 2048, 3500 and 4095 and the resulting out values and expected output voltages are recorded: mode 0 gives 1000, 2048, 3500, 4095; mode 1 gives 500, 1024, 1750, 2047; mode 2 gives 1000, 2048, 3102, 3102.

**Code and Output:**






Watch 1

| Name | Value | Type |
|---|---|---|
| mode | 0x00 | uchar |
| raw | 0x1000 | ushort |
| out | 0x1000 | ushort |
| <Enter expression> | | |

Watch 1

| Name | Value | Type |
|---|---|---|
| mode | 0x01 | uchar |
| raw | 0x1000 | ushort |
| out | 0x0800 | ushort |
| <Enter expression> | | |

Watch 1

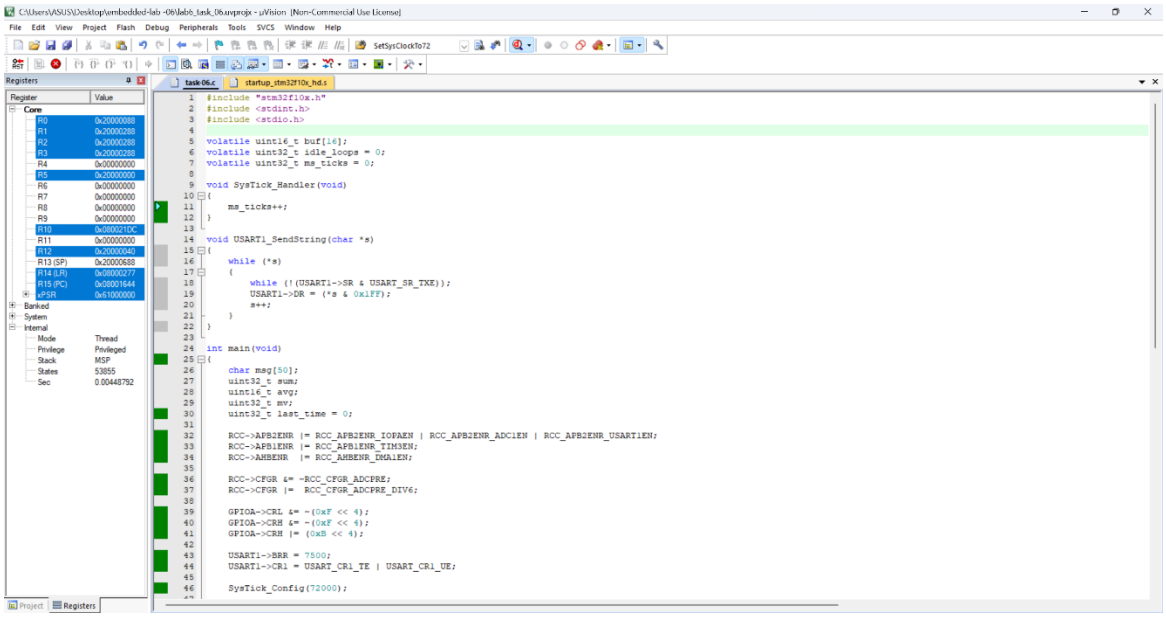
| Name | Value | Type |
|---|---|---|
| mode | 0x02 | uchar |
| raw | 0x0DAC | ushort |
| out | 0x0C1E | ushort |
| <Enter expression> | | |

**Challenge Task – Timer-triggered ADC + DMA + UART**

**Objective**: To build a TIM3-triggered 100 Hz ADC1/DMA acquisition chain that averages 16 samples and reports the result over USART1, demonstrating a low-CPU-load data path.

**Description:** USART1 is initialised as in Chapter 5 (9600 baud, 8N1, PA9 TX) with a USART1_SendString() helper, TIM3 is configured for 100 Hz (PSC = 7199, ARR = 99, TRGO update via CR2 = (2 << 4)), and ADC1 channel 1 is set for timer-triggered DMA operation (CR2 = 0x00180101) and calibrated. DMA1 Channel 1 fills volatile uint16_t buf[16] with CNDTR = 16 and CCR = 0x5A1, and the main loop waits about 500 ms (SysTick millis() or a simple counter) while incrementing idle_loops. Every 500 ms the average of the 16 entries and mv = avg × 3300 / 4095 are computed, formatted with sprintf(msg, "ADC=%4u mV=%4u\r\n", avg, mv) and sent; the ~18-character message takes about 18–20 ms at 9600 baud with 10 bits per character. Buffer values 0, 2048 and 4095 are forced to verify the messages ADC= 0 mV= 0, ADC=2048 mV=1650 and ADC=4095 mV=3300, and the chain TIM3 → ADC1 → DMA1 → memory → CPU → USART1 → PC is drawn and explained in terms of reduced CPU load compared with polling.

**Code and Output:**



```
#include "stm32f10x.h"
#include <stdint.h>
#include <stdio.h>

volatile uint16_t buf[16];
volatile uint32_t idle_loops = 0;
volatile uint32_t ms_ticks = 0;

void SysTick_Handler(void)
{
    ms_ticks++;
}

void USART1_SendString(char *s)
{
    while (*s)
    {
        while (!(USART1->SR & USART_SR_TXE));
        USART1->DR = (*s & 0x1FF);
        s++;
    }
}

int main(void)
{
    char msg[50];
    uint32_t sum;
    uint16_t avg;
    uint32_t mv;
    uint32_t last_time = 0;

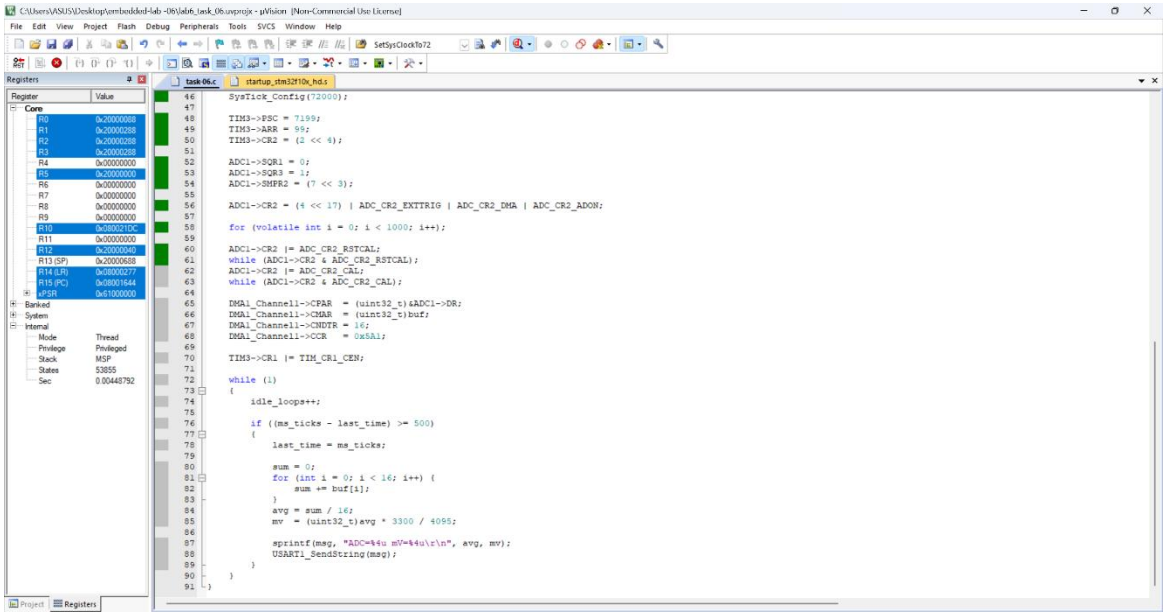
    RCC->APB2ENR |= RCC_APB2ENR_IOPAEN | RCC_APB2ENR_ADC1EN | RCC_APB2ENR_USART1EN;
    RCC->APB1ENR |= RCC_APB1ENR_TIM3EN;
    RCC->AHBENR  |= RCC_AHBENR_DMA1EN;

    RCC->CFGR &= ~RCC_CFGR_ADCPRE;
    RCC->CFGR |=  RCC_CFGR_ADCPRE_DIV6;

    GPIOA->CRL &= ~(0xF << 4);
    GPIOA->CRH &= ~(0xF << 4);
    GPIOA->CRH |= (0xB << 4);

    USART1->BRR = 7500;
    USART1->CR1 = USART_CR1_TE | USART_CR1_UE;

    SysTick_Config(72000);
```



```
    SysTick_Config(72000);

    TIM3->PSC = 7199;
    TIM3->ARR = 99;
    TIM3->CR2 = (2 << 4);

    ADC1->SQR1 = 0;
    ADC1->SQR3 = 1;
    ADC1->SMPR2 = (7 << 3);

    ADC1->CR2 = (4 << 17) | ADC_CR2_EXTTRIG | ADC_CR2_DMA | ADC_CR2_ADON;

    for (volatile int i = 0; i < 1000; i++);

    ADC1->CR2 |= ADC_CR2_RSTCAL;
    while (ADC1->CR2 & ADC_CR2_RSTCAL);
    ADC1->CR2 |= ADC_CR2_CAL;
    while (ADC1->CR2 & ADC_CR2_CAL);

    DMA1_Channel1->CPAR  = (uint32_t)&ADC1->DR;
    DMA1_Channel1->CMAR  = (uint32_t)buf;
    DMA1_Channel1->CNDTR = 16;
    DMA1_Channel1->CCR   = 0x5A1;

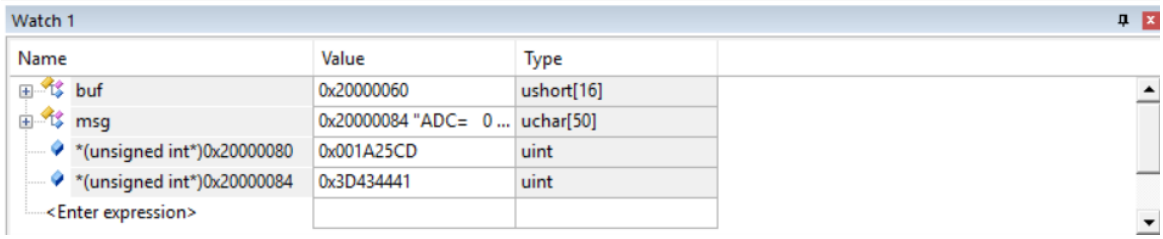
    TIM3->CR1 |= TIM_CR1_CEN;

    while (1)
    {
        idle_loops++;

        if ((ms_ticks - last_time) >= 500)
        {
            last_time = ms_ticks;

            sum = 0;
            for (int i = 0; i < 16; i++) {
                sum += buf[i];
            }
            avg = sum / 16;
            mv  = (uint32_t)avg * 3300 / 4095;

            sprintf(msg, "ADC=%4u mV=%4u\r\n", avg, mv);
            USART1_SendString(msg);
        }
    }
}
```



Watch 1

| Name | Value | Type |
|---|---|---|
| buf | 0x20000060 | ushort[16] |
| msg | 0x20000084 "ADC= 0 ... | uchar[50] |
| *(unsigned int*)0x20000080 | 0x001A25CD | uint |
| *(unsigned int*)0x20000084 | 0x3D434441 | uint |
| <Enter expression> | | |

**The Data Chain:**

- **TIM3 (Timer):** Acts as a precise hardware clock, generating a trigger pulse exactly at 100 Hz.
- **ADC1 (Analog-to-Digital Converter):** Receives the trigger from TIM3 and instantly starts converting the analog voltage on PA1 into a 12-bit digital value.
- **DMA1 (Direct Memory Access):** Takes the converted data from the ADC data register and moves it directly into the RAM (buf array) one by one, 16 times.
- **Memory (RAM):** Stores the 16 continuous ADC samples silently in the background.
- **CPU (Processor):** Wakes up every 500 ms, reads the 16 values from memory, calculates the average, and formats the string.
- **USART1 (Serial Communication):** Takes the formatted string from the CPU and transmits it to the PC at a 9600 baud rate.
- **PC (Computer):** Displays the final ADC and millivolt (mV) readings.

**How CPU load is reduced**

In this architecture, the CPU load is drastically reduced because it does not have to continuously poll the ADC or waste instructions copying data. The TIM3 and DMA1 handle the exact timing and data transfer completely in the background via hardware. The CPU only intervenes for a brief moment every 500 milliseconds to calculate the average and send the UART message, leaving it free to perform other critical tasks (which is proven by the increasing idle_loops counter).

**Conclusion**

This lab provided a comprehensive hands-on experience with the analog and digital interfaces of the STM32F103RC microcontroller. By configuring the ADC in single, continuous, and multi-channel scan modes, I learned how to accurately convert analog signals into 12-bit digital values and observed the practical implementation of sampling times. Furthermore, integrating the DMA controller demonstrated how to efficiently transfer multi-channel ADC data directly to memory without CPU intervention, significantly reducing processor load. The DAC experiments successfully showed how to generate precise analog voltages using both software triggers and buffer configurations. Finally, combining the ADC, CPU processing, and DAC in a single loop, along with the challenge task involving timer-triggered ADC, DMA, and USART transmission, highlighted how these peripherals work together in real-time embedded systems to process and monitor physical signals efficiently.